

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1703

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Design Task

Code of Conduct:

I E.Ragul / 192571032 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E. Ragul

Design Task

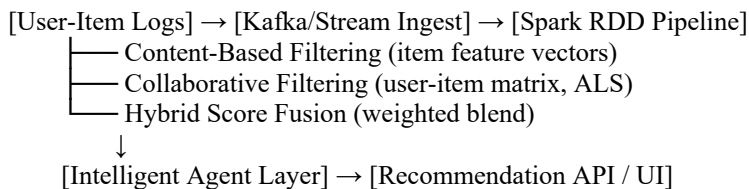
1. Hybrid AI Recommendation Engine Design

Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

ANSWER:

1.1 Architecture Overview

A three-layer pipeline: **Ingestion Layer** (clickstream, purchase logs, ratings, item metadata) → **Spark Processing Layer** (RDD-based hybrid model computation) → **Agent Layer** (personalization, feedback, and delivery agents).



1.2 RDD-Based Distributed Processing

- **Ingestion RDD:** raw interaction logs (user_id, item_id, rating, timestamp) are loaded as `sc.textFile()` and parsed into key-value RDDs.
- **Content-Based Filtering:** item metadata (genre, tags, description embeddings) is vectorized (TF-IDF or embeddings) and stored as an RDD of (item_id, feature_vector). Cosine similarity between vectors is computed via a cartesian() or broadcast-join pattern, avoiding full shuffle where possible.
- **Collaborative Filtering:** the user-item matrix is built as a sparse RDD of (user_id, item_id, rating) and fed into Spark MLlib's **ALS (Alternating Least Squares)**, which is natively RDD-based and distributes matrix factorization across partitions.
- **Hybrid Fusion:** `join()` combines content-based scores and CF-predicted scores per (user_id, item_id) key; a `map()` applies a weighted-average or learned blending function (weights tunable per user segment via `reduceByKey()`).
- **Partitioning strategy:** hash-partition by user_id to co-locate a user's interaction history, minimizing shuffle during personalization.

1.3 Role of Intelligent Agents

- **Profile Agent:** continuously updates a user's latent preference vector as new interactions arrive (incremental/online learning).
- **Feedback Agent:** monitors implicit (clicks, dwell time) and explicit (ratings, likes) signals, feeding a reward signal back into the fusion weights — effectively a lightweight reinforcement-learning loop (multi-armed bandit style exploration/exploitation).
- **Context Agent:** adjusts recommendations for session context (time of day, device, current browsing intent).
- **Orchestrator Agent:** arbitrates between content-based and collaborative signals when one is sparse (e.g., cold-start users get more content-based weight; heavy users get more CF weight).
-

1.4 Real-Time Adaptation & Scalability

Structured Streaming (micro-batches over RDD/DataFrame abstractions) ingests new events continuously; agents recompute preference vectors incrementally rather than retraining ALS from scratch each cycle (periodic full retrain + continuous incremental scoring). Horizontal scaling is achieved by adding Spark executors as the user base grows; recommendation serving is decoupled via a caching layer (e.g., Redis) fed by the batch/streaming Spark jobs.

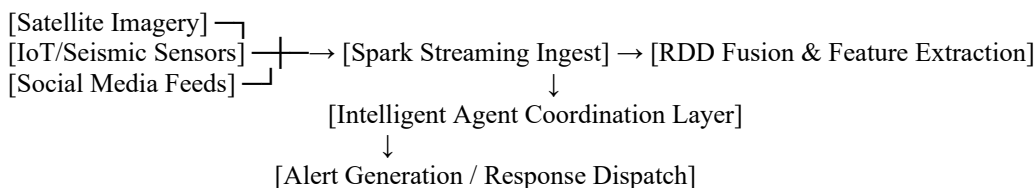
2.AI-Based Fraud Detection Framework

Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

Answer:

2.1 Architecture Overview

A multi-source, fusion-driven pipeline for early warning:



2.2 RDD-Based Distributed Processing

- Each data source is ingested as a separate RDD stream: imageRDD (tile_id, pixel_matrix), sensorRDD (station_id, reading, timestamp), socialRDD (post_id, text, geotag, timestamp).
- **Transformations:** map() extracts features (e.g., NDVI/flood indices from imagery, magnitude/frequency from seismic readings, keyword/sentiment scores from social text); filter() discards low-confidence or stale records; flatMap() expands multi-region posts into per-region records.
- **Geospatial partitioning:** RDDs are partitioned by geographic grid cell (geohash) so all data relevant to a region co-locates on the same executor, minimizing cross-node shuffle during fusion.
- **reduceByKey / aggregateByKey:** per-region signals from all three sources are aggregated into a composite "risk score" per grid cell per time window.

•

2.3 Data Fusion Techniques

- **Late fusion:** independent per-source anomaly scores (image-based flood extent, sensor threshold breach, social-media event-burst detection) are computed separately, then combined via a weighted or Bayesian fusion rule keyed by region and time window.
- **Temporal alignment:** join() on (region, time_bucket) keys synchronizes asynchronous sources (satellite passes are infrequent, sensors are continuous, social posts are bursty).
- **Confidence weighting:** sources with historically higher precision (e.g., sensors) are weighted more heavily than noisier sources (e.g., social media), with weights adapted over time based on validated outcomes.

2.4 Role of Intelligent Agents

- **Detection Agents** (one per data modality): specialize in interpreting their source (image-anomaly agent, seismic-signal agent, social-signal agent).
- **Fusion/Coordinator Agent:** aggregates detection-agent outputs, resolves conflicting signals, and decides whether the composite risk crosses an alert threshold.
- **Response Agent:** once a threshold is crossed, determines response strategy (evacuation zone, severity tier, notification channels) and dispatches alerts to relevant authorities/citizens.
- **Learning Agent:** after each event, agents update their confidence weights and thresholds based on ground-truth outcomes (was it a real disaster or false positive), improving future fusion accuracy.

2.5 Real-Time Adaptation & Scalability

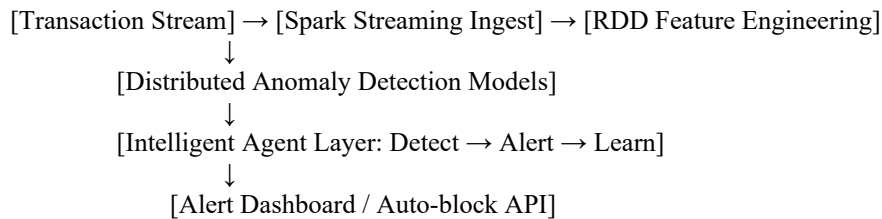
Spark Structured Streaming processes micro-batches (seconds-level latency) for sensor/social data; heavier image-processing jobs run in a slightly longer batch cadence and merge into the same fused risk-score RDD/DataFrame. The system scales by adding executors per data-source pipeline independently, and agents run as lightweight stateful services that subscribe to the fused output stream.

3. Autonomous Supply Chain Optimization System

Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

ANSWER:

3.1 Architecture Overview



3.2 RDD-Based Distributed Processing

- Transactions arrive as (txn_id, user_id, amount, merchant, location, timestamp) and are loaded into RDDs partitioned by user_id, co-locating each user's transaction history for fast behavioral comparison.
- **Feature engineering via transformations:** map() derives per-transaction features (deviation from user's average spend, velocity of transactions in a time window, geographic distance from last known location); groupByKey/aggregateByKey computes rolling user-level statistics (mean, std-dev, transaction frequency) needed for anomaly scoring.
- **Distributed anomaly detection:** statistical methods (Z-score, Mahalanobis distance) and ML methods (Isolation Forest, or streaming k-means for clustering "normal" behavior) are applied per-partition in parallel; Spark MLlib's RDD-based algorithms distribute both training and scoring across the cluster.

3.3 Intelligent Agent Collaboration

- **Monitoring Agents:** each watches a transaction stream partition (e.g., per region or per merchant category) and scores incoming transactions against the current anomaly model.
- **Alerting Agent:** aggregates high-score anomalies across monitoring agents, applies rule-based suppression (avoid duplicate alerts for the same user in a short window), and escalates to human review or auto-blocks based on severity.
- **Investigation-Feedback Agent:** ingests analyst verdicts (fraud confirmed / false positive) and feeds them back as labeled training data.
- **Model-Update Agent:** periodically retrains the anomaly detection model using accumulated feedback (active/online learning), redistributing the updated model to monitoring agents — this closes the loop for continuously improving detection accuracy.

3.4 Real-Time Adaptation & Scalability

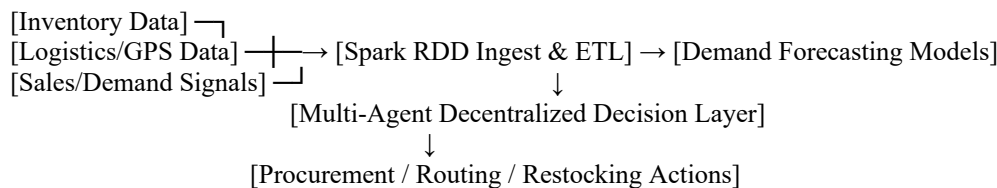
Structured Streaming with micro-batch windows (sub-second to few-second latency) supports near-real-time scoring; stateful streaming (mapWithState/updateStateByKey-style operations) maintains rolling per-user behavioral profiles without reprocessing full history each batch. The cluster scales horizontally by transaction volume, and the anomaly model can be updated incrementally rather than requiring full retraining, keeping detection current against evolving fraud patterns.

4. Personalized Learning Analytics Platform

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

ANSWER:

4.1 Architecture Overview



4.2 RDD-Based Distributed Processing

- Inventory, logistics, and sales data streams are each loaded as RDDs keyed by (warehouse_id/sku) or (vehicle_id/route), enabling parallel per-entity processing.
- **Demand forecasting:** historical sales RDDs are transformed via map()/reduceByKey() into time-series aggregates per SKU per region, which feed into distributed forecasting models (e.g., Spark MLlib regression, or time-series models run per-partition using mapPartitions() for efficiency).
- **Logistics optimization:** route and delivery data are processed with flatMap()/groupByKey() to compute per-vehicle load, distance, and time-window feasibility; results seed a distributed optimization routine (e.g., a parallelized heuristic for vehicle routing).
- **Join across domains:** join() combines forecasted demand with current inventory levels per SKU/warehouse to compute a restocking-need RDD, which becomes the shared state multiple agents act upon.

4.3 Role of Multiple Intelligent Agents (Decentralized Decisions)

- **Demand Forecast Agent:** predicts near-term demand per region/SKU and publishes updated forecasts.
- **Inventory Agent** (one per warehouse): monitors local stock levels against forecast and decides when to trigger replenishment, without needing central approval for routine restocks.
- **Logistics Agent** (one per route/fleet segment): optimizes delivery schedules and vehicle loads based on current orders and constraints (fuel, time windows), negotiating with other logistics agents when routes overlap.
- **Procurement Agent:** negotiates supplier orders based on aggregated replenishment signals from inventory agents, balancing cost minimization against lead-time risk.

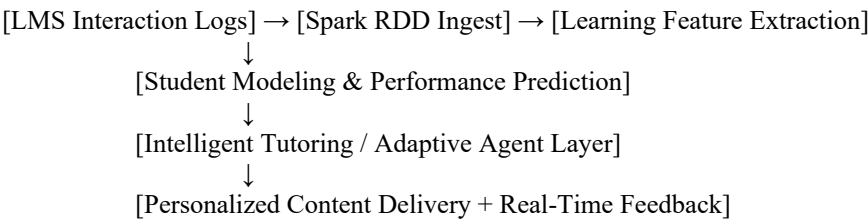
5. Personalized Learning Analytics Platform

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning

through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners

ANSWER:

5.1 Architecture Overview



5.2 RDD-Based Distributed Processing

- Learner interaction logs (student_id, activity_id, timestamp, score, time_spent, attempts) are ingested as RDDs partitioned by student_id, so each learner's full activity trace is processed together.
- **Feature extraction:** map()/mapPartitions() compute per-student features (mastery level per topic, engagement trends, error patterns); reduceByKey() aggregates performance across sessions into a rolling competency profile.
- **Cohort-level analytics:** groupByKey()/aggregateByKey() on topic or class-level keys support instructor dashboards showing class-wide mastery distributions, computed in parallel across thousands of learners.
- **Predictive modeling:** Spark MLlib (RDD-based classification/regression) predicts at-risk students or likely next-best content, trained on the aggregated feature RDDs.

5.3 Intelligent Agents for Adaptive Learning

- **Student Modeling Agent:** maintains each learner's evolving knowledge-state model (e.g., mastery per skill/topic), updated incrementally with every new interaction.
- **Content Recommendation Agent:** selects the next learning activity/resource based on the student's current knowledge gaps and preferred learning pace/style.
- **Feedback Agent:** delivers real-time hints/corrections during exercises and adjusts difficulty on the fly (adaptive difficulty scaling) based on immediate performance signals.
- **Early-Intervention Agent:** flags students showing disengagement or persistent struggle patterns, alerting instructors and adjusting the content path proactively (e.g., inserting remedial material).

5.4 Real-Time Adaptation & Scalability

Streaming ingestion of interaction events allows the Student Modeling Agent to update mastery estimates within seconds of an activity submission, enabling immediate adaptive content selection for the next exercise. The platform scales to thousands of concurrent learners by partitioning all processing by student_id (embarrassingly parallel per-learner computation), with periodic batch jobs handling heavier cross-student analytics (cohort trends, model retraining) separately from the low-latency per-student adaptive loop.

Common Design Pattern Across All Five Systems

Design	How It's Realized in PySpark/RDDs	Agent Role
--------	-----------------------------------	------------

Element		
Ingestion	sc.textFile() / Structured Streaming from multiple sources	Source-specific monitoring agents
Feature extraction	map(), flatMap(), mapPartitions()	Modeling/profile agents
Aggregation	reduceByKey(), groupByKey(), aggregateByKey()	Fusion/coordinator agents
Cross-source fusion	join() on shared keys (user, region, SKU, student)	Orchestrator agents
Distributed ML	Spark MLlib (ALS, clustering, regression)	Prediction/decision agents
Real-time loop	Structured Streaming micro-batches + incremental state updates	Feedback/learning agents
Scalability	Partitioning aligned to natural entity keys (user/region/SKU/student) to minimize shuffle	Horizontal agent replication per partition

This shared pattern shows how the same distributed-computing primitives (RDD transformations, partitioning-by-key, and Spark's fault-tolerant lineage model) generalize across very different domains, while the **intelligent agent layer** is what turns raw distributed computation into an adaptive, decision-making system — sensing state, reasoning over it, acting, and learning from the outcome.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation